17-03-2026

IAM - Identity and Access Management.

An **IAM User** is an **identity in AWS** that represents a **person** that needs to **access AWS resources**.
Each user has **a name, credentials, and permissions** that define what they can and cannot do in your AWS account.
Best practice:

- Don't use the **root user** for daily work.
- Create **individual IAM users** for each person or service.
- Use **least privilege** (only give needed permissions).

You can group users using **IAM Groups** to manage permissions easily.

Global Service

- IAM is **not regional**:
- You create users, roles, and policies **once** and they work across all AWS regions.
- Contrast with S3 buckets, EC2 instances, which are **region-specific**.

Why It Exists (Problems It Solves)

1. **Stop sharing root account** — too dangerous.
2. **Fine-grained control** — let DevOps start/stop EC2, block Finance from deleting.
3. **Temporary credentials for apps** — EC2 can talk to S3 without hard-coding passwords.
4. **Auditability** — know **who did what** (CloudTrail uses IAM identities).
5. **Security compliance** — enforce least privilege, MFA, key rotation.

Root User

- **Created automatically** when you first open an AWS account.
- Has **full permissions for everything** (billing, deleting account, creating IAM).
- **Use only for initial setup** (MFA, creating first admin user).

- **Do NOT use for daily work.**
- Too powerful, no separation of duties.

IAM Users

- **Individual identity** for a person or service that needs long-term access.
- Has **credentials**:
- Console: username/password + MFA.
- Programmatic: Access key ID & secret.

Groups
Groups let you attach a policy once and add/remove users easily

- **Container for users** that share the same permissions.
- You attach **policies** to the group, not each user.
- Users **inherit** all policies from groups they're in.

Organize by job function (Admins, DevOps, ReadOnly).

POLICY:
Policy in AWS IAM is a document (written in JSON format) that defines permissions — it tells who can do what actions on which resources in your AWS account.

policy = set of rules that control access to AWS services.

Policy Structure:

- **JSON documents** that define **Allow** / **Deny** for **Actions** on **Resources**.

Bucket policy to allow public access when block public access has been disabled at all levels:

```
{
  "Version": "2012-10-17",
```

```
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::your-bucket-name/*"
    }
  ]
}
```

Mandatory parts:

- **Effect:** Allow or Deny
- **Action:** API calls (e.g., s3:GetObject)
- **Resource:** What the actions apply to
- **Version:** Specifies the policy language version (usually "2012-10-17").
- **Statement:** Main block where you define one or more permission rules.

Optional fields:

- **Sid** – Statement ID (for your reference).
- **Condition** – Extra conditions (IP ranges, MFA, etc.).
- **Principal** – Required only when the policy defines *who* is allowed (like in S3 bucket policies, assume role policies).

Effect:

Default = deny everything (implicit deny).
Allow = grants the action.
Explicit Deny = always overrides any allow.

Quick scenario:

- Group policy allows s3:*
- You add user policy: Deny s3:DeleteObject
- → User **cannot delete** even though group allows.

- **Policy Types:**

AWS Managed Policies

- Pre-built by AWS for common roles.
- Auto-updated by AWS when new services/features are added.
- AdministratorAccess — full access to everything.
- PowerUserAccess — everything except IAM and billing.
- AmazonS3ReadOnlyAccess — read-only to all S3.
- AmazonEC2FullAccess — manage EC2.

Customer Managed Policies

- You create and maintain.
- Great when you need custom least-privilege.
- Can be shared across users/groups/roles in your account.

Inline Policy:
Inline policy is **attached directly *inside* a user, group, or role.**

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::dev-bucket"
    }
  ]
}
```

AWS best practice:
Use **managed policies** whenever possible (for reusability & easy maintenance).
Use **inline policies** only when you need permissions unique to one specific identity.
Managed preferred over Inline for reuse and easier maintenance.

Resource-Based Policy?

- A **resource-based policy** is a JSON policy that is **attached directly to an AWS resource** rather than to a user, group, or role.

If your policy has "Principal" — it's **resource-based** or a **trust policy**, not identity-based.
If your policy doesn't have "Principal" — it's **identity-based** (can be inline or managed).

Resource-Based Policies Exist

- Sometimes **the owner of the resource** wants to control who can access it directly.
- Useful when:
- You want to **share a resource across AWS accounts**.
- A service needs to call into your resource.
- You want fine-grained control at the object itself.

Make an S3 Bucket Public or Shareable
Service: Amazon S3
You attach a **bucket policy** directly to your bucket to control access.
Example:
Allow everyone (Principal: "*") to read objects in your bucket.

```
{
  "Version": "2012-10-17",
  "Statement": [
```

```
    {
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-website-bucket/*"
    }
  ]
}
```

Role:

We use role in 3 scenarios:

1) For few resources like EC2, we cannot attach policy directly. So we create a role, attach policy to this role & attach that role to EC2.

Ex: EC2 connecting with S3

2) When we want to **grant temporary access to a guest user** to an account, we use role.
3) To facilitate **cross account access**, we use role.

Role Policies: Trust vs Permissions:

Trust Policy = who can assume; Permissions Policy = what can do.

Best Practices

- **Use groups for human users.**
- **Attach managed or customer-managed policies to groups** (not directly to each user).
- Keep **inline policies** rare.
- Have a separate **Admin group** with AdministratorAccess for a few trusted people.
- Enable **MFA** for high-privilege groups.